Materia: Programación IV

UNIDAD 6 · RELACIONES Y ESTADO

APUNTE TEÓRICO

Arquitectura relacional y relaciones 1:N con FastAPI y SQLModel

Revisor de la unidad: Prof. Juan Ignacio Sarmiento

1. Definición Conceptual del Modelo de Dominio

El fundamento de cualquier base de datos relacional sólida comienza por entender cómo se vinculan las entidades.

La Relación 1:N (Uno a Muchos): Implica que un único registro del lado "uno" tiene la capacidad de estar asociado a múltiples registros del lado "muchos". El ejemplo canónico del dominio es un equipo (Team) que posee varios héroes (Hero).

La Perspectiva N:1 (Muchos a Uno): Es el principio fundamental de que la relación 1:N y la N:1 son exactamente la misma relación, pero analizada desde ángulos opuestos. Un héroe pertenece a un equipo.

El Principio de la Clave Foránea: La regla de oro, inquebrantable en el modelo relacional, dicta que la clave foránea (FK) siempre debe residir en la tabla correspondiente al lado "muchos".

Aislamiento del Padre: El registro padre (lado "uno") jamás almacena referencias de sus entidades hijas a nivel de estructura de base de datos. No necesita saber cuántos registros dependen de él para existir de forma independiente como tabla.

2. Implementación de la Capa de Persistencia y ORM

SQLModel exige una división estricta entre lo que se guarda en el disco (SQL) y cómo se interactúa en memoria (Python).

2.1. El Lado "Muchos" (Hero) - Estructura Física

Este modelo es el responsable de declarar las claves foráneas que impactan en la base de datos.

```python
# Lado "Muchos" con la FK real
class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    # Esta línea genera: team_id INTEGER REFERENCES team(id)
    team_id: int | None = Field(default=None, foreign_key="team.id")
```

Traducción SQL: Esta instrucción genera directamente una restricción en la base de datos, equivalente a `team_id INTEGER REFERENCES team(id)`.

El campo se define típicamente como opcional (`int | None`) para indicar que un héroe puede existir sin estar afiliado a un equipo.

2.2. Navegación en Memoria (Relationship)

La función `Relationship()` es una construcción puramente del ORM y bajo ningún concepto genera columnas nuevas ni altera las tablas mediante un `ALTER TABLE` en la base de datos.

Su propósito exclusivo es permitir la navegación fluida entre objetos relacionados en Python una vez que estos han sido cargados en la memoria.

Sincronización Bidireccional: El uso del parámetro `back_populates` es estrictamente obligatorio para mantener la consistencia en memoria. Si se modifica `hero.team`, el ORM actualiza automáticamente `team.heroes`.

```python
class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    # Navegación hacia los hijos
    heroes: list["Hero"] = Relationship(back_populates="team")

class Hero(SQLModel, table=True):
    # ... campos anteriores ...
    # Navegación hacia el padre. back_populates DEBE ser simétrico
    team: Optional["Team"] = Relationship(back_populates="heroes")
```

3. Arquitectura de Exposición de Datos (Schemas)

Exponer el modelo persistente (`table=True`) al cliente es un fallo de seguridad y diseño gravísimo. Se deben usar esquemas de Pydantic.

Control de Payload: Los esquemas definen de manera granular qué datos exactos se devuelven en la respuesta HTTP.

Esquemas de Lectura Básica (HeroRead, TeamRead): Exponen únicamente los campos propios de la entidad y las claves foráneas crudas (ej. `team_id`), sin anidar objetos pesados.

Esquemas de Lectura Completa (HeroReadFull, TeamReadFull): Extienden los esquemas básicos para incluir las relaciones completas. Por ejemplo, `HeroReadFull` incluye atributos opcionales para weapon (tipo `WeaponRead`) y team (tipo `TeamBasicRead`).

Mitigación de Dependencias Circulares: Es una decisión de diseño crítica. Para evitar que el módulo de equipos importe al de héroes y viceversa (generando un bucle), cada módulo debe definir y exportar un esquema muy reducido del otro. `HeroBasicRead` se define en `team/schemas.py`, y `TeamBasicRead` en `hero/schemas.py`.

```python
# En hero/schemas.py
class TeamBasicRead(SQLModel):  # Schema reducido para romper el bucle
    id: int
    name: str

class HeroReadFull(HeroRead):  # Lectura completa con anidados
    team: Optional[TeamBasicRead] = None
```

4. Estrategia de Rendimiento: Carga de Datos y el Problema N+1

Este es un punto crítico donde una implementación cuidadosa marca la diferencia, ya que un error de carga puede colapsar los servidores.

Comportamiento por Defecto (Lazy Loading): SQLModel emplea carga diferida; los objetos relacionados no se buscan en la base de datos hasta el instante en que el código intenta acceder al atributo (ej. `hero.team`).

Riesgo 1: Error de Sesión: Si la sesión de la base de datos ya se cerró al momento de acceder al atributo de relación, la aplicación lanza un error fatal.

Riesgo 2: El Problema N+1: Si se lista una colección de "N" héroes y se accede a la relación de equipo para cada uno, el ORM ejecuta una consulta inicial más "N" consultas adicionales, destruyendo la eficiencia. En SQL, un acceso lazy se traduce como un `SELECT * FROM team WHERE id = ?` ejecutado repetitivamente.

Resolución Arquitectónica (Eager Loading): Para endpoints que exigen devolver datos anidados, es obligatorio configurar el servicio para realizar una carga anticipada utilizando `selectinload`. Esto resuelve el problema ejecutando una única segunda consulta altamente eficiente antes de que la sesión finalice.

```python
from sqlalchemy.orm import selectinload

def get_hero(hero_id: int, session: Session):
    # Se obtiene el héroe y su equipo
    statement = select(Hero).where(Hero.id == hero_id).options(
        selectinload(Hero.team)
    )
    return session.exec(statement).first()
```

5. Flujo de Ciclo de Vida y Persistencia

- **Creación (Request):** Para asociar un recurso hijo a su padre, el cliente solo envía la FK (ej. `team_id`) en el payload del esquema de creación (`HeroCreate`). No hace falta cargar todo el objeto Team en memoria.
- **Validación (Servicio):** La capa de servicio tiene la obligación innegociable de verificar que el registro padre exista realmente antes de ejecutar la persistencia.
- **Persistencia y Respuesta:** Se guarda la FK en la tabla y, solo si el esquema de respuesta lo demanda, se serializa el objeto relacionado anidado.
- **Garantía de Integridad (DB):** Si la validación falla a nivel de servicio, la base de datos lanza un `IntegrityError` al intentar insertar una FK inexistente o al intentar borrar un padre que tiene hijos asignados, a menos que se configure explícitamente un comportamiento en cascada (CASCADE).

```python
@app.post("/heroes/", response_model=HeroRead)
def create_hero(hero: HeroCreate, session: Session = Depends(get_session)):
    # 1. Validar existencia del padre
    if hero.team_id:
        db_team = session.get(Team, hero.team_id)
        if not db_team:
            raise HTTPException(status_code=404, detail="Team not found")
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero
```

Bibliografía

1. FastAPI. (s. f.). FastAPI documentation. Recuperado el 2 de octubre de 2026, de https://fastapi.tiangolo.com
2. PostgreSQL Global Development Group. (s. f.). PostgreSQL documentation. Recuperado el 2 de octubre de 2026, de https://www.postgresql.org/docs
3. Pydantic. (s. f.). Pydantic documentation. Recuperado el 2 de octubre de 2026, de https://docs.pydantic.dev

4. SQLModel. (s. f.). SQLModel documentation. Recuperado el 2 de octubre de 2026, de https://sqlmodel.tiangolo.com